

Reliability of AI-generated code and Security Challenges

*Guest Speaker Talk: Flinders University,
Adelaide, Australia*

Dr. Ahmad Mohsin, Research Fellow

[a.mohsin@ecu.edu.au]

School of Science, Edith Cowan University, Australia



Discussion points

- Software and Security
- Large Language Models. AI Agents
- Code Generation with LLMs
- In-Context Security Learning for LLMs
- Security Testing for LLMs Code
- AI-Driven Software Engineering
- Key Takeaway
- Conclusion

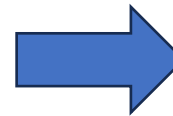
A simple Programming Example

Write a Simple C++ function that reads user input into a character array and prints the Entered text by user on screen.

```
#include <iostream>

void readInput() {
    char buffer[100];
    std::cout << "Enter some text: ";
    std::cin >> buffer;
    std::cout << "You entered: " << buffer << std::endl;
}

int main() {
    readInput();
    return 0;
}
```



Program output will look like this:

Enter some Text: We are going work with LLMs

You entered : We are going to work LLMs

Generated by AI Model

Input

Write a Simple C++ function that reads user input into a character array and prints the Entered text by user on screen.



Magic

Large Language Model (GPT)



Output- Generated Code

```
#include <iostream>

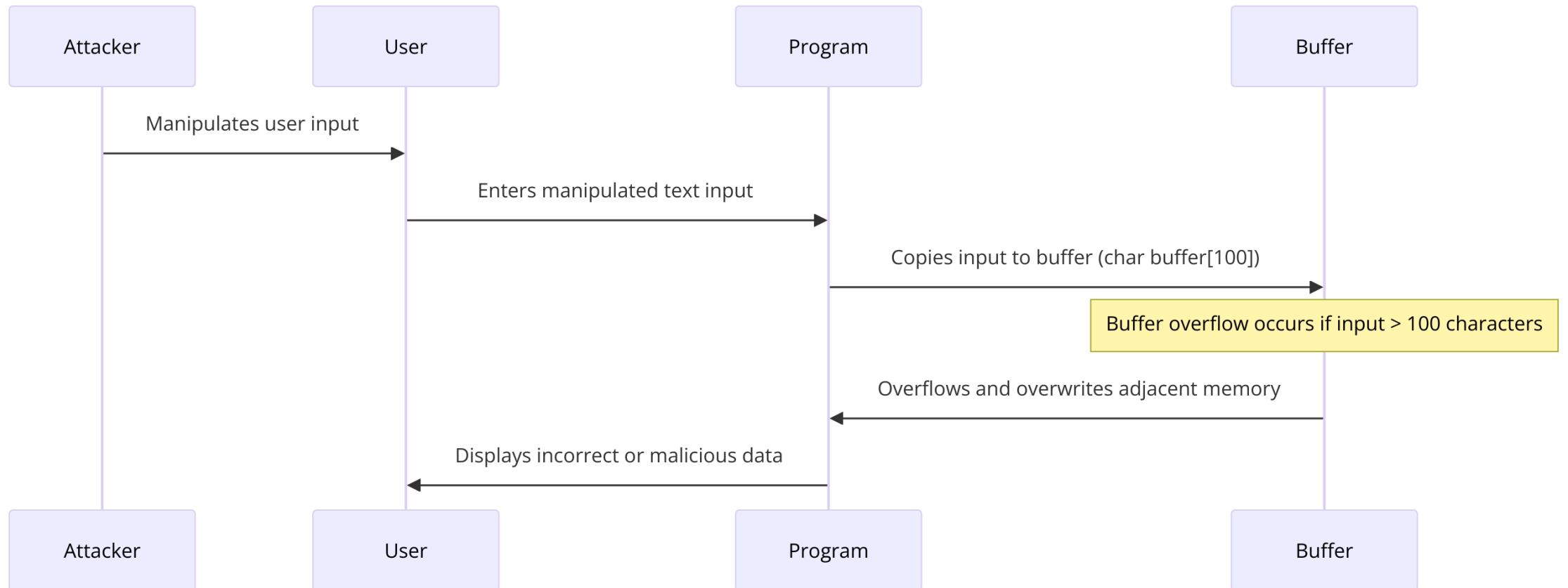
void readInput() {
    char buffer[100];
    std::cout << "Enter some text: ";
    std::cin >> buffer;
    std::cout << "You entered: " << buffer << std::endl;
}

int main() {
    readInput();
    return 0;
}
```

Observed Vulnerabilities in Code

vulnerability Description	Line of Code Involved	CWEs	Potential Cyber Attacks (with CVEs)
Buffer Overflow	<code>std::cin >> buffer;</code> (Line 6)	CWE-120: Classic Buffer Overflow	Buffer overflow attacks can lead to arbitrary code execution, denial of service, and memory corruption. Examples include CVE-2015-7547 in the GNU C Library and similar vulnerabilities in various software that allow remote attackers to execute arbitrary code or cause a denial of service
Unbounded Input Read	<code>std::cin >> buffer;</code> (Line 6)	CWE-787: Out-of-bounds Write	This kind of vulnerability is often exploited in ways similar to buffer overflow, where excessive input data without proper bounds checking leads to overwrite critical memory areas. No specific CVE for this code, but similar patterns have been exploited in numerous systems.
Information Disclosure	<code>std::cout << "You entered: " << buffer << std::endl;</code> (Line 7)	CWE-200: Information Exposure	If buffer overflows and contains sensitive information or system memory due to improper handling, printing out the buffer can lead to information leakage. No direct CVEs related to this exact scenario, but the risk of disclosing potentially sensitive memory contents inadvertently is a concern.

Attack through Buffer overflow



Software Vulnerabilities and Secure by Design

Vulnerability

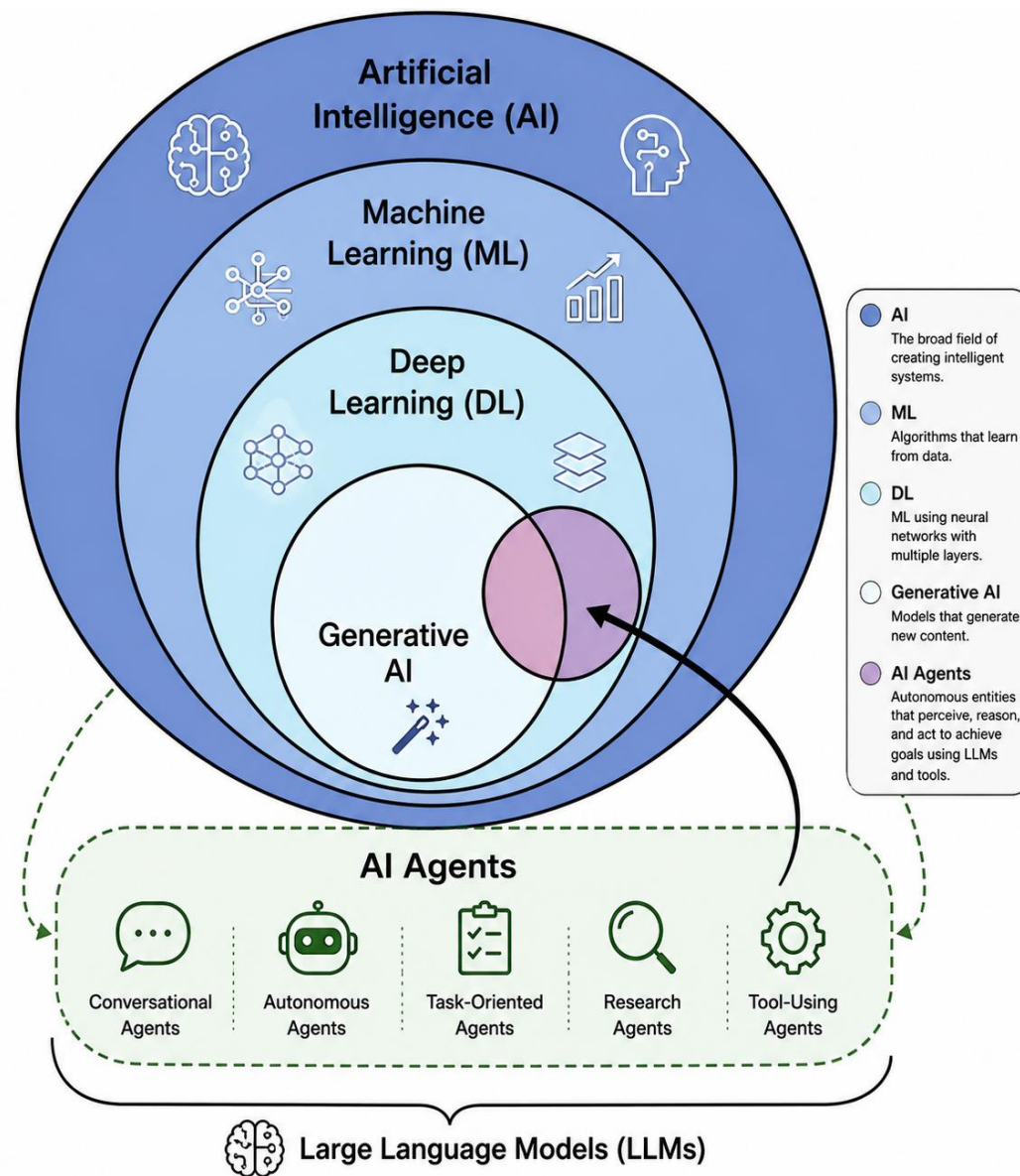
- A security flaw, glitch, or weakness found in software code that could be exploited by an attacker. (NIST)

CWEs

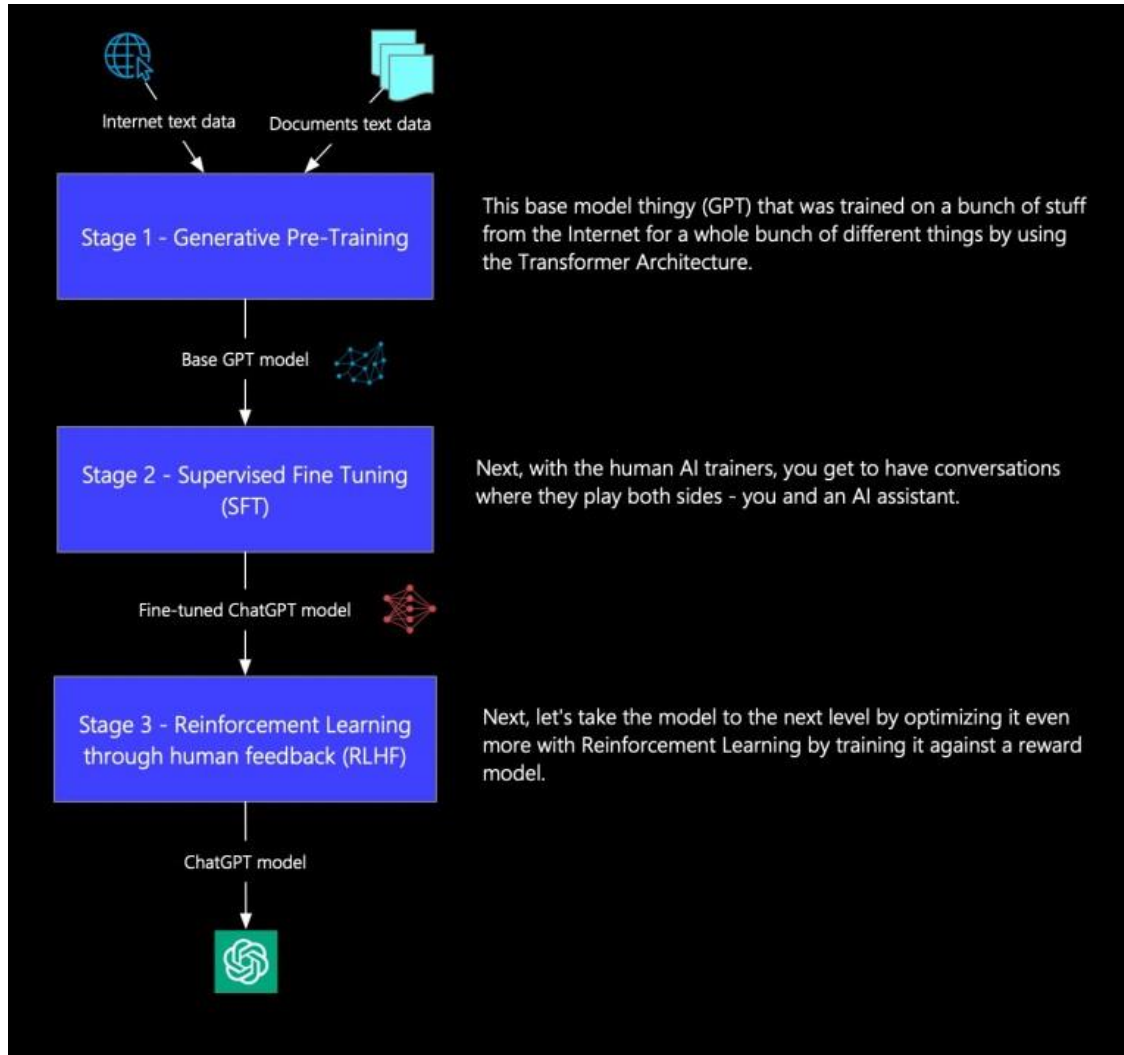
- The Common Weakness Enumeration (CWE) is a category system for hardware and software weaknesses and vulnerabilities. It is sustained by a community project with the goals of understanding flaws in software and hardware and creating automated tools that can be used to identify, fix, and prevent those flaws.
- ***Secure by design*** is a challenge especially in the age of Automated Code Generation through AI

What is current AI?

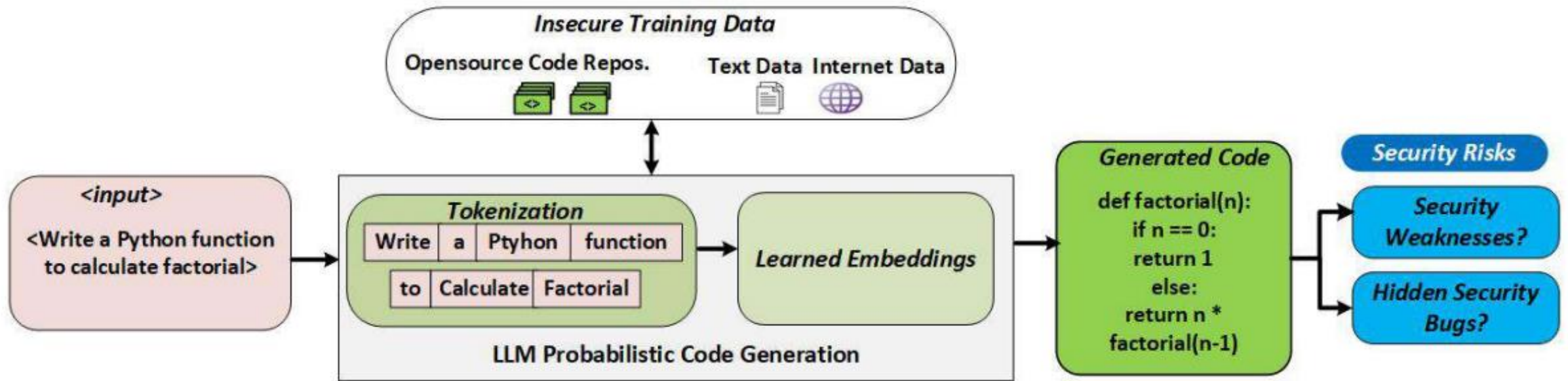
AI, ML, Deep Learning and Generative AI



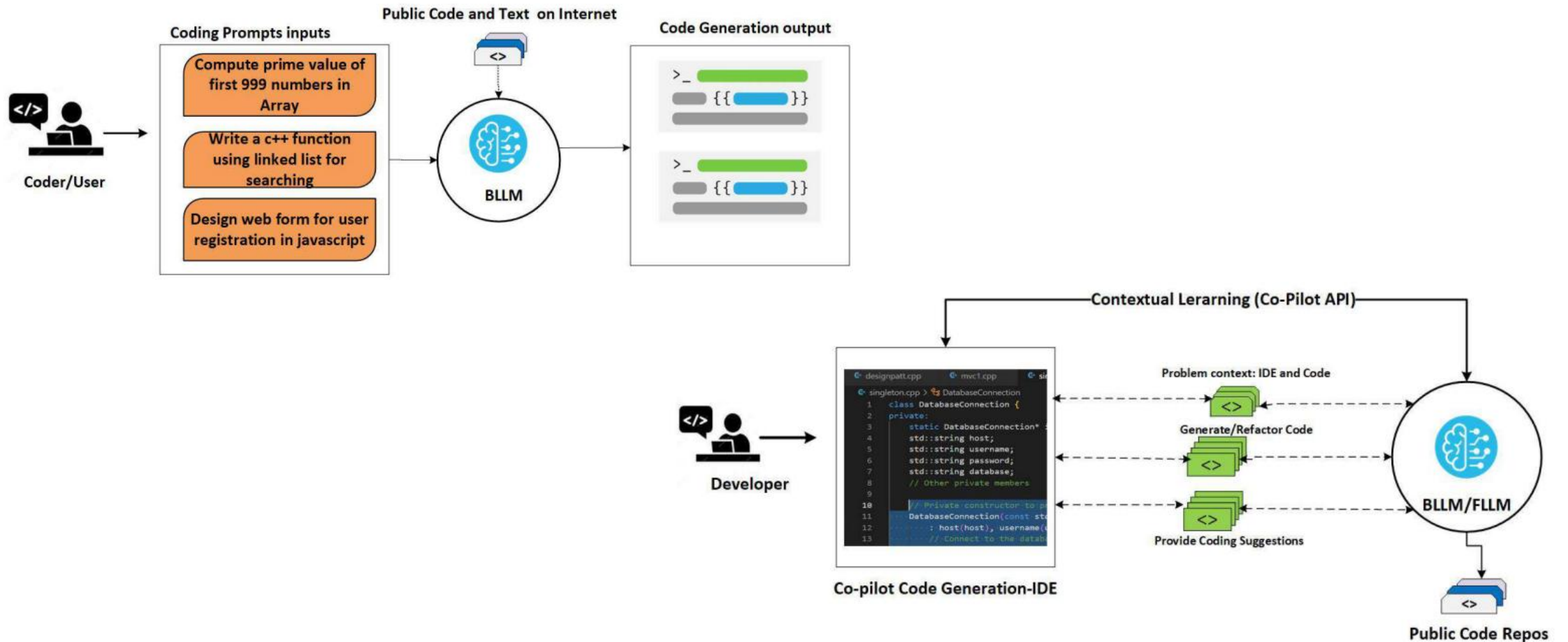
How LLMs are Trained



Code Generation with LLMs



Types of LLM Code Generators



Can we Enhance LLMs Learning capabilities?



FINE TUNING



RAG BASED
LEARNING



IN CONTEXT
LEARNING



CHAIN OF THOUGHT
REASONING

In-context Learning of LLMs

Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 cheese => ..... ← prompt
```

One-shot

In addition to the task description, the model sees a single example of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 sea otter => loutre de mer ← example
3 cheese => ..... ← prompt
```

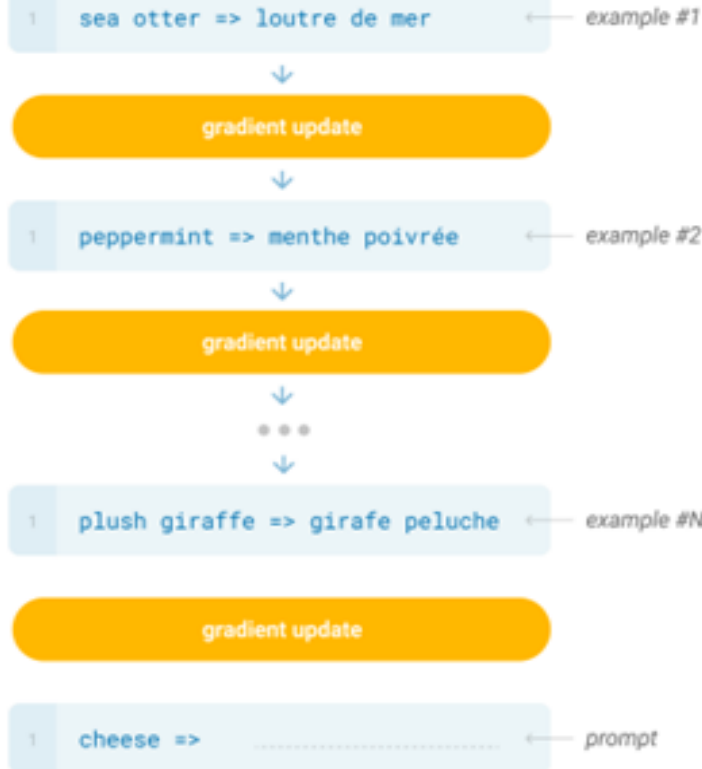
Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 sea otter => loutre de mer ← examples
3 peppermint => menthe poivrée ←
4 plush girafe => girafe peluche ←
5 cheese => ..... ← prompt
```

Fine-tuning

The model is trained via repeated gradient updates using a large corpus of example tasks.



In-Context Learning with Chain of Thought Reasoning

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

(a) Few-shot

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A:

(Output) The answer is 8. ❌

(b) Few-shot-CoT

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A:

(Output) The juggler can juggle 16 balls. Half of the balls are golf balls. So there are $16 / 2 = 8$ golf balls. Half of the golf balls are blue. So there are $8 / 2 = 4$ blue golf balls. The answer is 4. ✅

(c) Zero-shot

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A: The answer (arabic numerals) is

(Output) 8 ❌

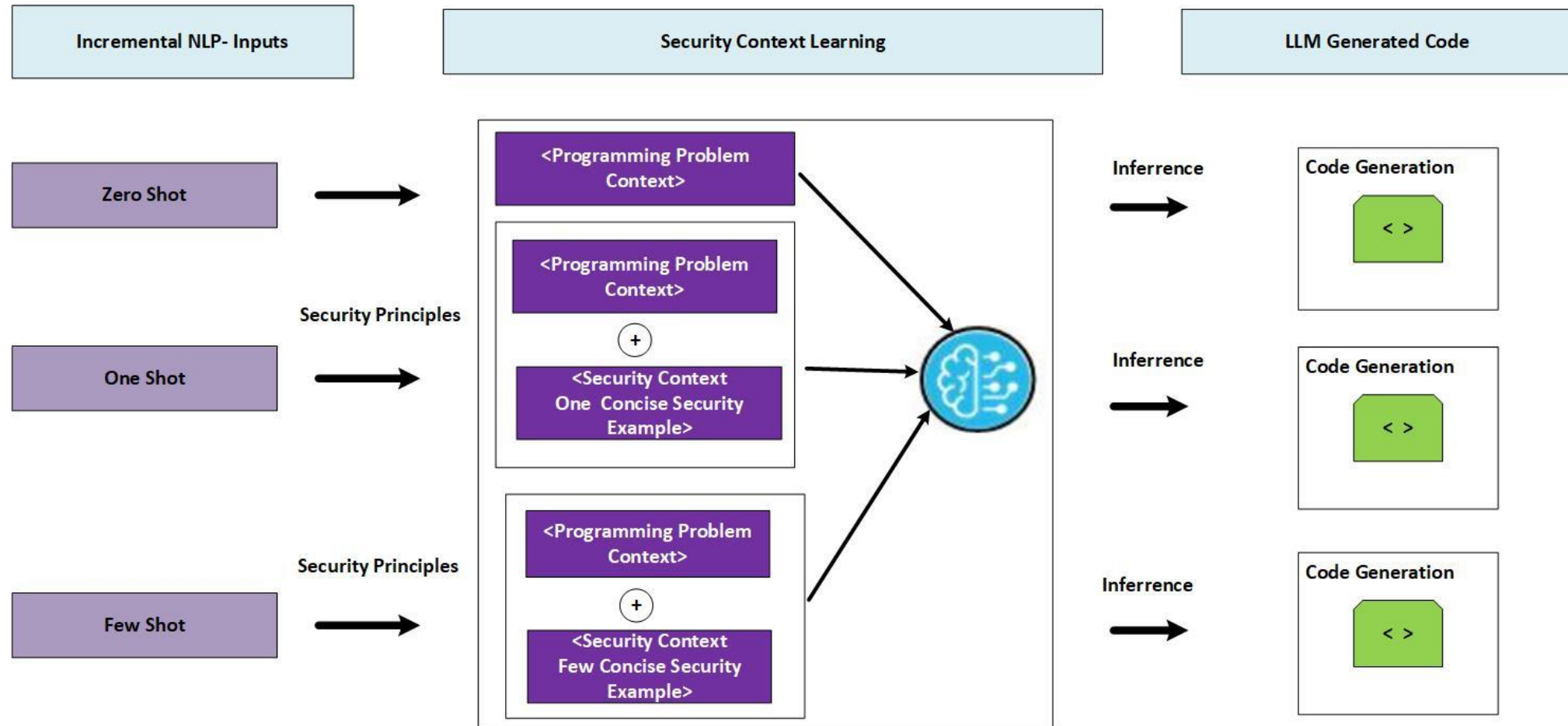
(d) Zero-shot-CoT (Ours)

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A: **Let's think step by step.**

(Output) There are 16 balls in total. Half of the balls are golf balls. That means that there are 8 golf balls. Half of the golf balls are blue. That means that there are 4 blue golf balls. ✅

In-Context Learning and Inference Generation



In-Context Learning Security Pattern-LLMs

In-Context Learning (ICL) Security Pattern for LLMs (PDCGs and CCPs)

ICL Security Pattern

Zero-Shot Learning

- **Description:** LLM Generates code based on pre-existing knowledge without specific examples on security.
- **LLM Input:** Generate a hash function to securely convert plain text into a fixed-size hash value, ensuring data integrity and security.

```
function hashFunction (password) {  
    return hash(password);}
```



Contextual Reasoning - LLMs

One-Shot Learning

- **Description:** LLM is provided with a single example to demonstrate secure coding practices.
- **LLM CoT-Reasoning:** The provided hash function is vulnerable because it doesn't include a salt, which makes it susceptible to rainbow table attacks.
- **LLM Input- One Security Example:** Develop a secure hash function using salt.

```
function secureHashFunction(password, salt) { // use salt  
    return hash(password + salt);}
```



Contextual Reasoning - LLMs

Few-Shot Learning

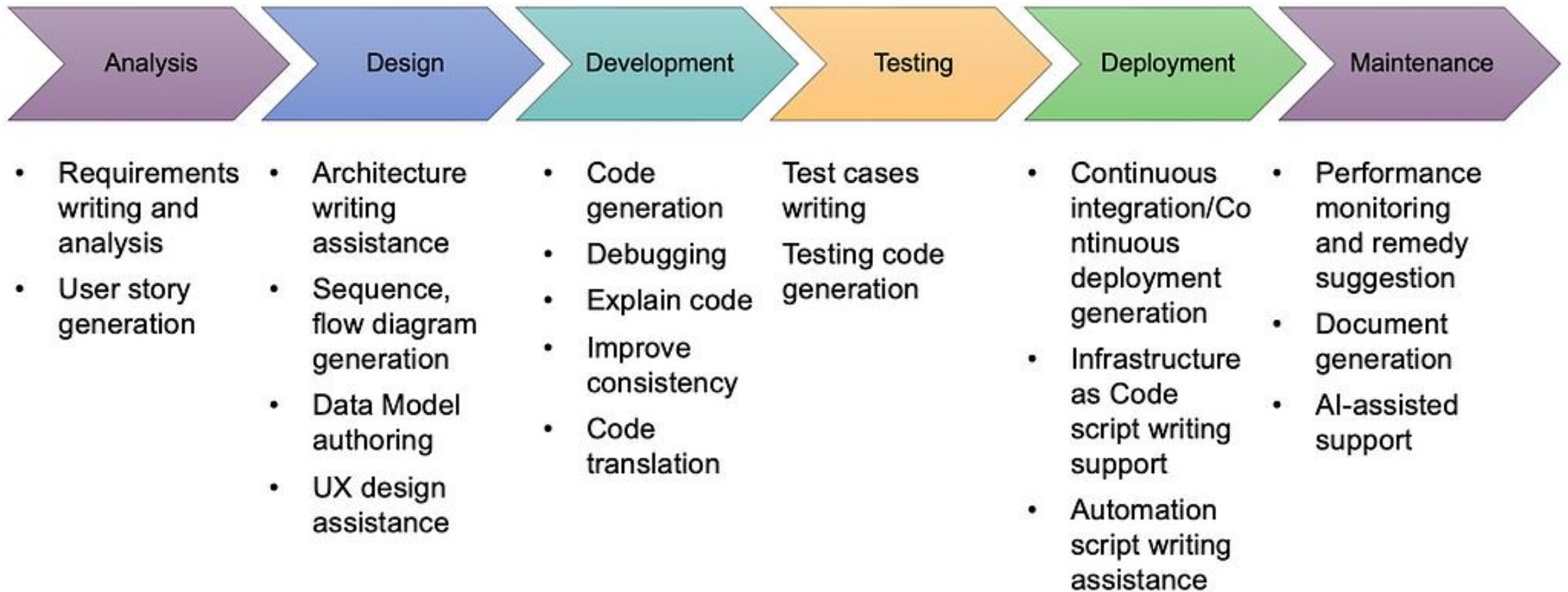
- **Description:** LLM receives multiple secure coding examples illustrating a range of security contexts.
- **LLM CoT-Reasoning:** Using only a salt is not sufficient for ensuring security. We need to use unique salts and other security measures like encryption and input validation.
- **LLM Inputs- Multiple Security Examples:**

```
// Use uniqueSalt with password  
function secureHashFunction(password, uniqueSalt) {  
    return hash(password + uniqueSalt);}  
function encryptData(data, key) {  
    return encrypt(data, key); } // Use AES with key expansion  
function validateUserInput(input) {  
    return sanitizeInput(input); } // Prevent SQL injections
```

ICL in Action

Chain of Thought Reasoning	In-Context Learning	LLM Generated Code
Secure Input Handling Using std::string Excess data can overwrite adjacent memory spaces, corrupting data or altering the flow of the program. Efficient memory management through Strings	std::string instead of a char array to prevent buffer overflow. String is suitable for memory dynamically, and how this choice enhances security against buffer overflow attacks.	<pre>#include <iostream> #include <string> void readInputSecurely() { std::string input; std::cout << "Enter some text: "; std::getline(std::cin, input); std::cout << "You entered: " << input << std::endl; }</pre> <pre>for (char c : input) { if (!isalnum(c)) { std::cout << "Invalid input. Only alphanumeric characters\n"; return; } }</pre>
read input, which safely handles input of any size.	Input size function check to ensure string size is not violated	
Inputs are fine, but you should ensure no characters are allowed other than alphanumeric.	Develop input validation check function as void inputvalidatin() for during user input	<pre>#include <exception> void readInputWithErrorHandling() { std::string input; try { std::cout << "Enter some text: "; std::getline(std::cin, input); std::cout << "You entered: " << input << std::endl; } catch (const std::exception& e) { std::cerr << "Error occurred: " << e.what() << std::endl; } }</pre>
Add try-catch blocks to handle exception	Add I/O error Handling Try { catch Blocks}	

LLMs in Software Engineering



LLMs usage for Software Development



DATA STRUCTURES AND
ALGORITHMS



WEB APPLICATIONS/
MOBILE APPLICATIONS



APIS DEVELOPMENT

LLM Code Generation with ICL- Google Bard

Security Context- One-Shot- Google Bard C++ Data Structures and Algorithms (Sorted Linked List)

The code you generated allocates memory for linked list nodes using new but doesn't explicitly free that memory. In the main function, it deallocates the memory, but it's essential to ensure that all memory allocated with new is properly deallocated with delete to avoid memory leaks. Please rewrite the code to fix the potential memory leak issue.



```
// Security Context: memory cleanup for merged list
void cleanupMemory(Node* head) {
    while (head) {
        Node* temp = head;
        head = head->next;
        delete temp;
    }
}

// Security Context: memory cleanup for list1 and list2
cleanupMemory(list1);
cleanupMemory(mergedList); // Deallocate memory for merged list

return 0;
}
```

Security Context- few-Shot- Google Bard C++ Data Structures and Algorithms (Sorted Linked List)

Perform Integers validation when reading from input files and ensure integrity checks. Protect from directory Traversal Attacks and ensure Error Handling is applied.



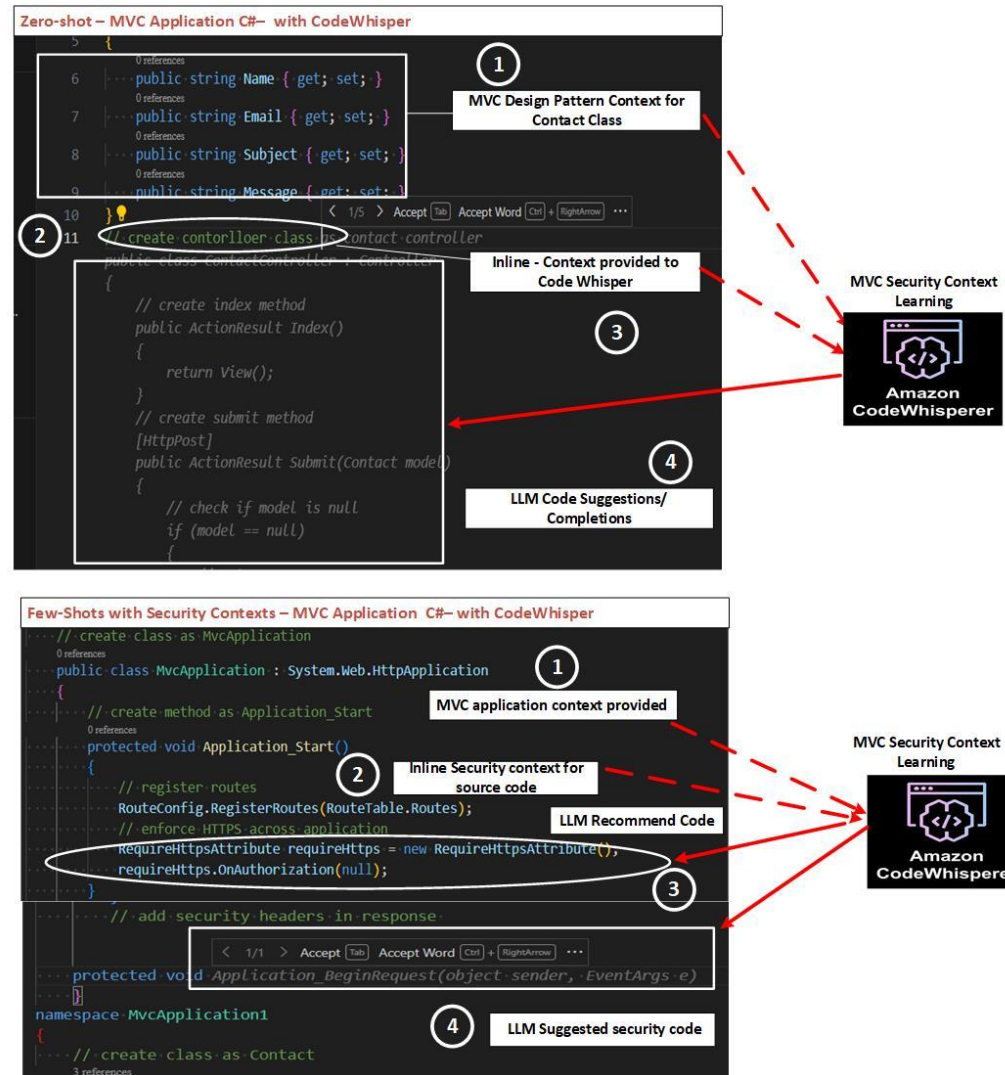
```
// Security Context- File Validation and ensure integrity checks.
bool isValidFile(string filename) {
    ifstream inputFile(filename);
    return inputFile.good();
}

// Security Context- Integers Validation when reading from input files
bool isValidInteger(string str) {
    stringstream ss(str);
    int num;
    if (ss >> num) {
        return ss.eof();
    } else {
        return false;
    }
}

// Security Context- Directory Traversal checks.
bool isValidDirectory(string directory) {
    return true;
}

// Security Context- Path Traversal check
bool isValidPath(string path) {
    return true;
}
```

ICL Code Generation with Code Whisperer



News API- LLM Zero Shot Description

A News API provides a wide range of functionalities to deliver timely and engaging news content to users. It offers user management, allowing users to register, authenticate, and manage their profiles. Users can access a diverse selection of news articles retrieved from various sources, filter content by topics, areas, and categories, and stay updated with real-time or near-real-time breaking news.



The API enhances user interactions with news articles, enabling comments, ratings, and reviews. It also offers advanced search capabilities, personalized news recommendations, and supports multimedia content, including images and videos. Social sharing options facilitate the dissemination of news articles across social media platforms. Users can access article summaries for quick overviews and receive notifications for new comments, interactions, and important news updates. Content syndication via RSS and Atom feeds is available for external applications. Analytics and insights are provided for news publishers to track article performance and audience demographics. Additionally, content licensing options and mechanisms for content verification help maintain content integrity and authenticity. Overall, the News API aims to deliver an engaging, informative, and customizable news consumption experience to users and applications.



Develop this News API in Python applying Flask Framework.

Functional Requirements for News API

1. User Management

- Users can register, authenticate, and manage their profiles.
- Ensure secure authentication and authorization mechanisms.

2. News Retrieval

- Provide endpoints to retrieve news articles, filter by topics, areas, and categories.
- Support real-time or near-real-time breaking news updates.

3. News Submission and Management

- Allow applications to create, update, and delete news articles.
- Include features for adding images, tags, and metadata to articles.

4. User Comments and Interactions

- Enable users to view, add, update, and delete comments on news articles.

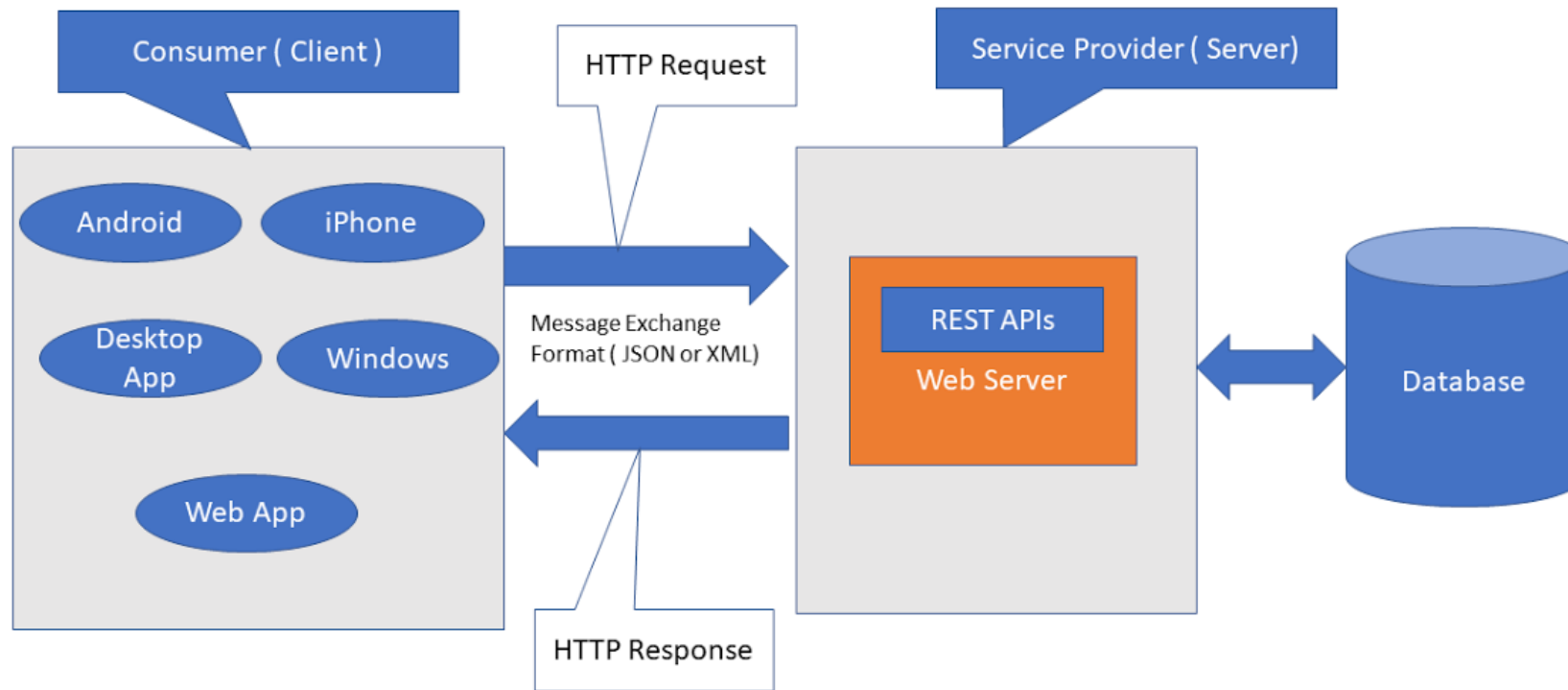
5. Search Functionality

- Implement advanced search capabilities for news articles based on user queries and filters.

6. User Notifications

- Provide notifications for new comments, interactions, and important news updates.

RESTful Architecture



Security Requirements-OWASP-ASVS

1. ****Authentication and Authorization:****

2. ****Rate Limiting:****

3. ****Input Validation and Sanitization:****

4. ****HTTPS Encryption:****

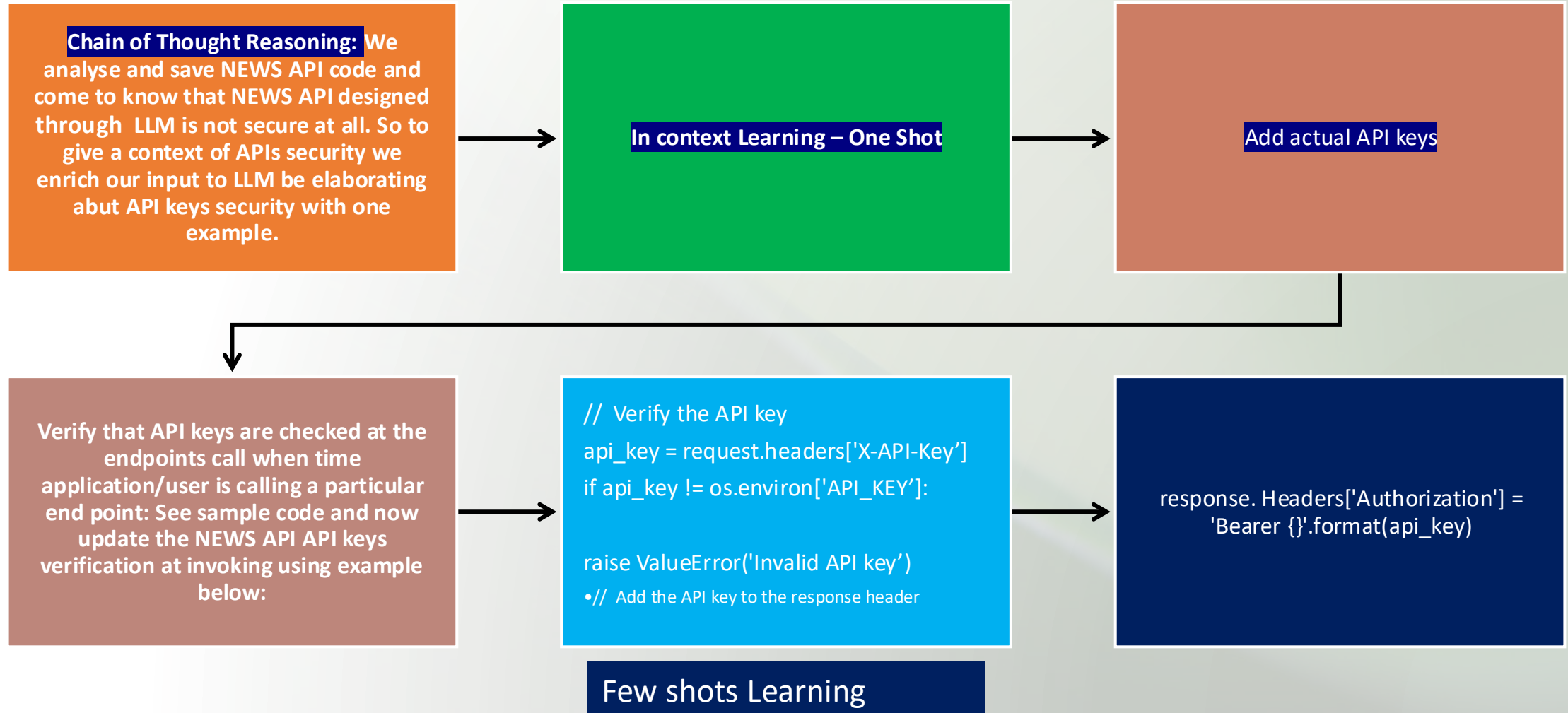
5. ****API Keys:****

6. ****Cross-Origin Resource Sharing (CORS):****

7. ****Authentication Tokens and Sessions:****

8. ****Security Headers:****

Train LLM with Security knowledge



LLM Code Generation with GitHub Copilot

One-Shot with Security Context – RESTful API Design Python- Github Co-pilot

Ensure that API keys are verified when endpoint is called for adding new news articles. update this code with this security aspect in existing Endpoint.

Accept Discard ↻

Security context to existing API keys code provided ①

```
87 # NewsAPI Endpoint to add new news articles
88 @app.route('/add-news', methods=['POST'])
89 def add_news():
90     # Get the news article data from the request
91     news_data = request.get_json()
92
93     # Get the API key from the request
94     api_key = request.args.get('api_key')
95
96     # Check if the API key is valid
97     if api_key != api_keys['newsapi']:
98         return jsonify({'error': 'Invalid API key'}), 401
99
100     # Connect to the AQL database
101     db = connect_to_aql_database()
102
103     # Create a new news article document
104     news_article = db.createDocument('news_articles', news_data)
105
106     # Save the document
107     news_article.save()
108
```

② LLM- API Keys verification code for Endpoints invocation suggested

GitHub Copilot

LLM- Behavior Learning with Natural Language and security principles inputs

Few-Shots with Security Contexts – RESTful API Design Python- Github Co-pilot

```
135 # NewsAPI Endpoint for Getting Top Headlines
136 @app.route('/top-headlines', methods=['GET'])
137 @auth.login_required
138 def top_headlines():
139     # Get the API key from the request
140     api_key = request.args.get('api_key')
141
142     # Check if the API key is valid
143     if api_key != api_keys['newsapi']:
144         return jsonify({'error': 'Invalid API key'}), 401
145
146     # Make the API request to the NewsAPI
147     url = f"https://newsapi.org/v2/top-headlines?country=us&apiKey={api_key}"
148     response = requests.get(url)
149     data = response.json()
150
151     # Return the top headlines
152     return jsonify(data)

```

① RESTful APIs Endpoints context

② LLM code suggestion for API keys

③ LLM Suggested code for authentication

```
# Add the API key to the response header for authorization
response = jsonify({'message': 'News article added successfully'})
response.headers['Authorization'] = api_key

return response

```

GitHub Copilot

Software Application Security Testing

SAST, which is short for Static Application Security Testing, is a security testing method that scans application source code or static binaries to uncover risks. The main purpose of SAST is to detect known vulnerabilities, such as application code or dependencies that are linked to vulnerabilities recorded as part of CVE databases.

SAST Tools : Snyk Security, CodQL



Discovered Vulnerabilities- Post Zero Shot

CWE ID	Description	Severity Level	Lines of Code Involved
CWE-200	Exposure of Sensitive Information to an Unauthorized Actor: The application returns detailed error messages or sensitive information, such as API key invalidation reasons, that could aid attackers in crafting further attacks.	Medium	abort(401, "Invalid API key")
CWE-89	SQL Injection: The code constructs SQL queries using parameters that are directly added without proper sanitization or prepared statements, potentially allowing an attacker to manipulate queries.	High	cursor.execute(sql, params) in multiple endpoints
CWE-306	Missing Authentication for Critical Function: The API uses a simple token-based authentication without validating the token's integrity or ensuring it's tied to a session or user entity securely.	Medium	@authenticate decorator and the authenticate(fn) function
CWE-311	Missing Encryption of Sensitive Data: The application does not encrypt sensitive data such as API keys stored in api_keys.txt, and potentially transmitted data, if not using HTTPS.	High	Storage: api_keys.txt loading process; Transmission: Not explicitly shown but implied by Flask setup
CWE-732	Incorrect Permission Assignment for Critical Resource: The application might improperly manage filesystem permissions on api_keys.txt, potentially exposing sensitive data.	Medium	API keys loading from api_keys.txt
CWE-759	Use of a One-Way Hash without a Salt: While not directly applicable since passwords or hashes aren't explicitly handled, the principle of not adding complexity or layers to security measures (like simple API key usage) can be a risk.	Medium	API key validation in authenticate(fn)
CWE-284	Improper Access Control: The application lacks detailed access control mechanisms beyond API key verification, potentially allowing unauthorized access to sensitive functions.	Medium	Entire application due to the simplistic authentication approach

Discovered Vulnerabilities- Post One Shot

CWE ID	Description	Severity Level	Lines of Code Involved
CWE-200	Exposure of Sensitive Information: Exposing detailed error messages (like "Missing query parameter") could potentially aid attackers. While not directly sensitive, careful phrasing of error messages is important.	Medium	abort(400, "Missing query parameter") and similar lines
CWE-89	SQL Injection: Although the code uses parameters (%s) in SQL queries which may mitigate direct SQL injection risks, using ORM methods or prepared statements is recommended for enhanced security.	Medium	SQL query construction and cursor.execute(sql, params)
CWE-306	Missing Authentication: The application uses a single API key for authentication, which could be insufficient for operations requiring different permission levels or detailed audit trails.	Medium	if api_key != API_KEY: in authenticate(fn)
CWE-319	Cleartext Transmission of Sensitive Information: If the application is not served over HTTPS, API keys and potentially sensitive information could be exposed in transit.	High	Entire application, Flask setup does not enforce HTTPS
CWE-798	Use of Hard-coded Credentials: Storing API keys in environment variables is good, but fallback to a hard-coded key ("YOUR_SECRET_API_KEY") could lead to security risks if not properly managed.	Medium	API_KEY = os.environ.get("API_KEY", "YOUR_SECRET_API_KEY")

Discovered Vulnerabilities- Post Few Shots

CWE-311	Missing Encryption of Sensitive Data: API keys and potentially sensitive information might be transmitted in clear text if HTTPS is not enforced.	High	API Key Transmission: <code>API_KEY = os.environ.get("API_KEY", "YOUR_SECRET_API_KEY")</code>
CWE-798	Use of Hard-coded Credentials: The fallback value "YOUR_SECRET_API_KEY" in the environment variable loading mechanism could pose a risk if the code is publicly exposed without proper management of default values.	Medium	<code>API_KEY = os.environ.get("API_KEY", "YOUR_SECRET_API_KEY")</code>
CWE-330	Insufficiently Random Values: Utilization of API keys without a secure generation and validation mechanism may lead to predictable or weak keys.	Medium	<code>import secrets</code> (implies generation but not shown)
CWE-89	SQL Injection: Although not directly shown, improper handling of SQL queries with user input could lead to SQL injection vulnerabilities.	High	Database interactions <code>cursor.execute(sql, params)</code>
CWE-306	Missing Authentication for Critical Function: The code demonstrates the use of decorators for authentication but lacks detailed implementation of token validation logic, potentially allowing unauthorized access.	High	<code>@auth.login_required</code> and <code>verify_token(token)</code>

How Different LLMs Learn: Security Knowledge

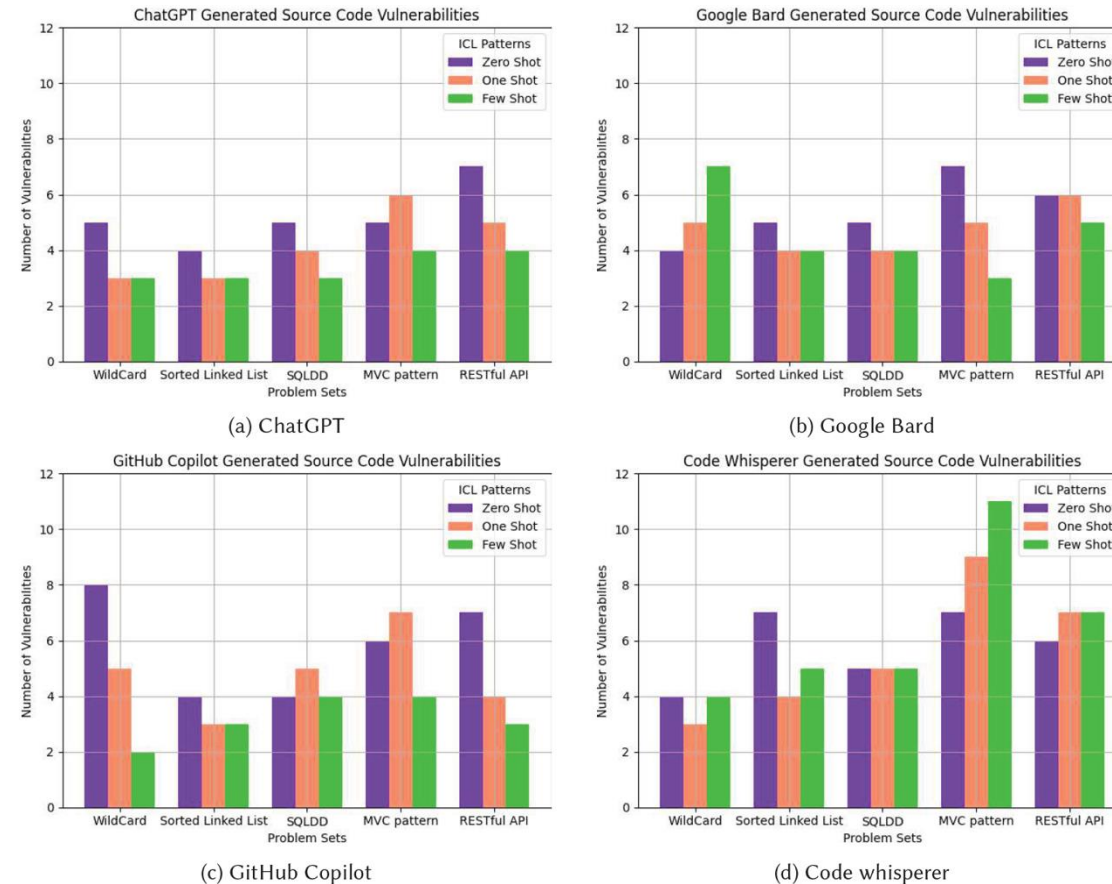


Fig. 8. Comparison of Vulnerabilities Generated by ChatGPT, Google Bard, GitHub Copilot, and Code whisperer

LLMs ability to Reduce Vulnerabilities

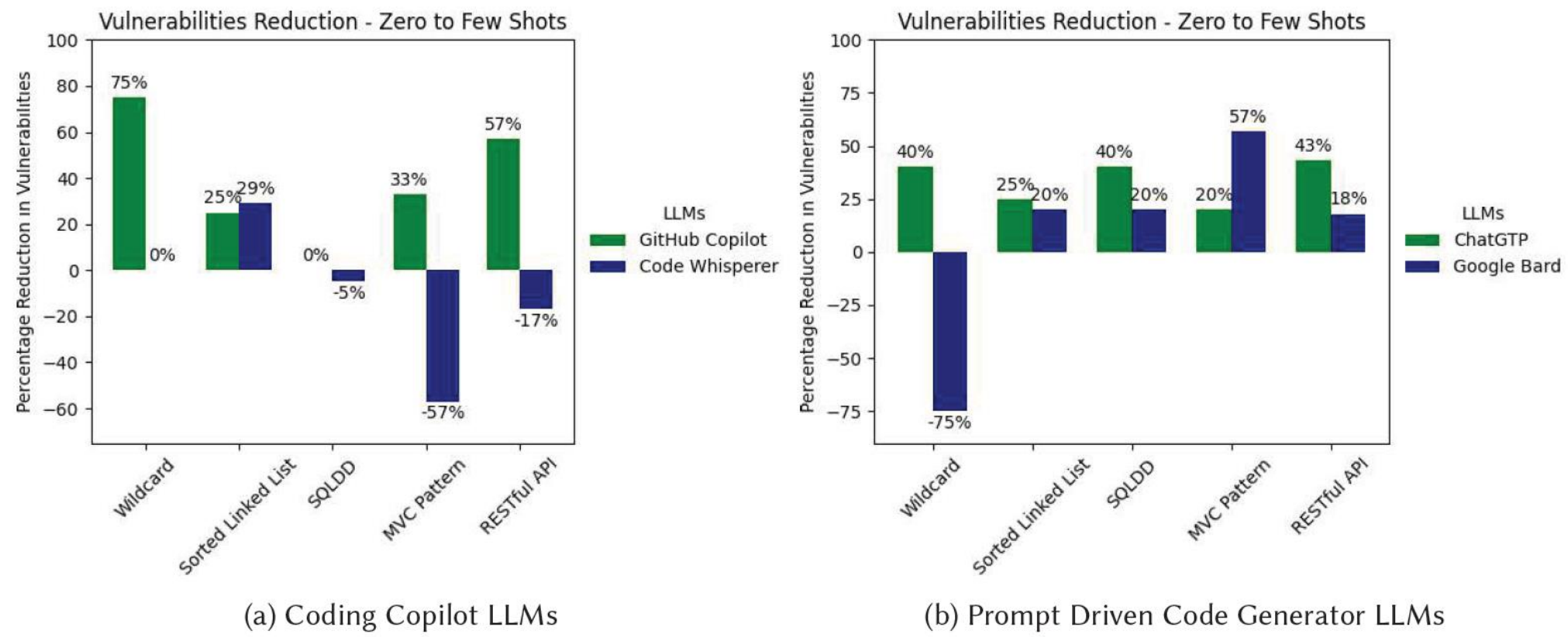
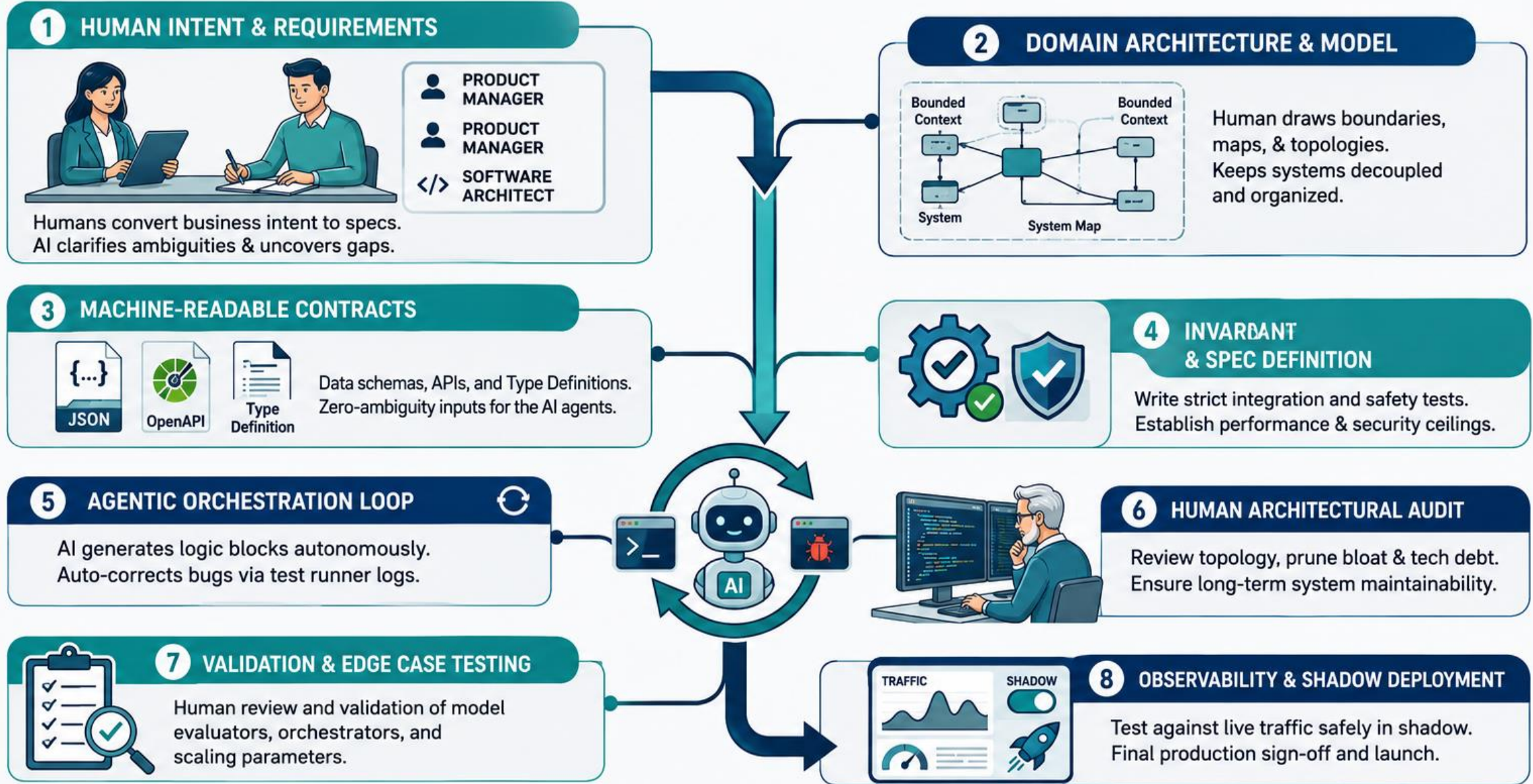
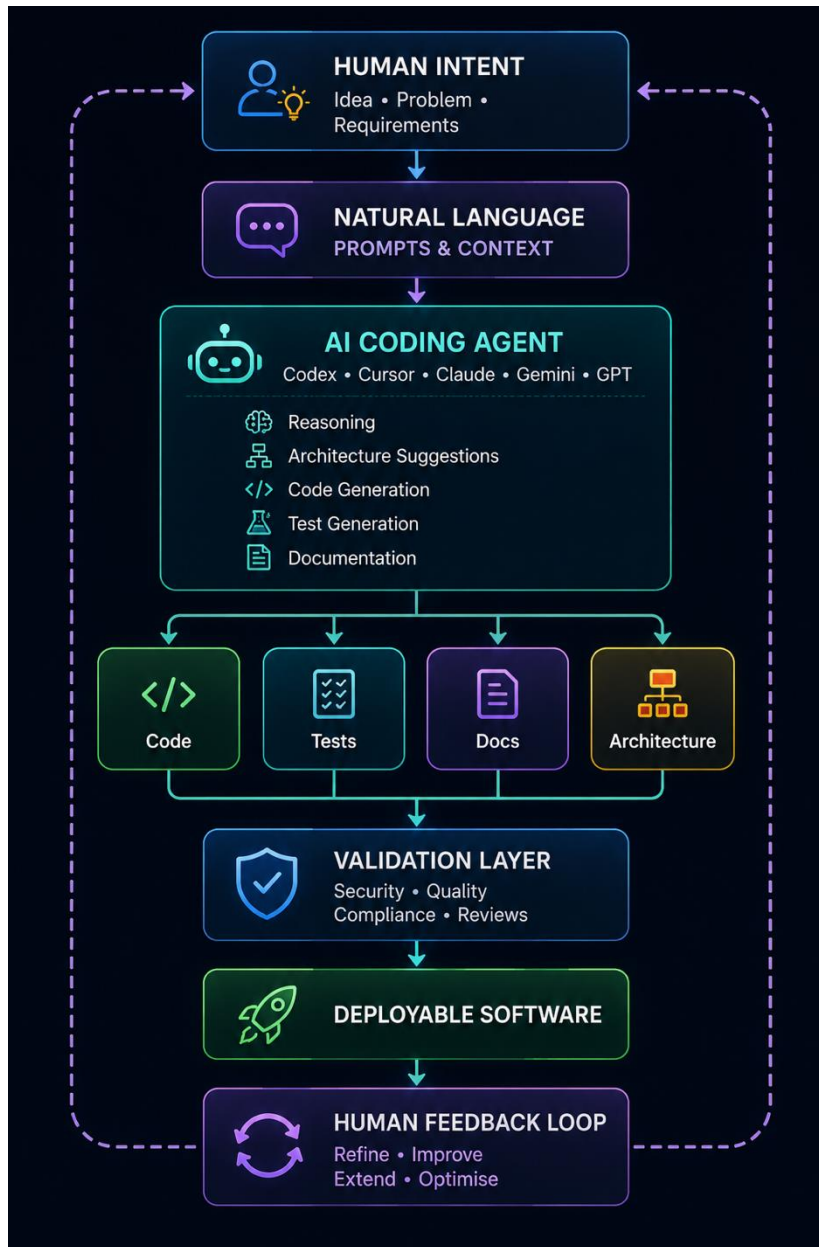


Fig. 10. LLMs Code Vulnerability Reduction Analysis for LLMs- ICL security patterns

THE AI-DRIVEN SOFTWARE ENGINEERING LIFECYCLE (AI-SDLC)



Future of Software Engineering- Vibe Coding?



Conclusion

- LLMs are good for Automated Code generation and enhance Developers Productivity
- However Code generated through LLMs is not always Safe and Secure ready for Deployment to Production lines
- LLMs can be implicitly Fine Tuned to provide Security knowledge using diverse set of Techniques
- Generated Code Must be refactored, tested, and reviewed before implementation.
- LLMs are Future for Automated Code Generation, however more research, innovation is needed to develop reliable and trustworthy systems of the future.

